

The Hong Kong Polytechnic University
Department of Computing

COMP4913 Capstone Project
Report (Interim)

**Compress-Transfer-Decompress for LLM
Serving: cuSZp-Enabled CPU–GPU Data
Pipeline in vLLM**

Student Name:	KONG Zirui
Student ID No.:	22103493D
Programme-Stream Code:	61435-FCS
Supervisor:	Dr ZHANG Zhaorui
Co-Examiner:	Prof. WU Yujie
2 nd Assessor:	Prof. YANG Hongxia
Submission Date:	09 January 2026

Abstract

This interim report presents the progress of integrating cuSZp compression library into the vLLM framework to optimize CPU-GPU data transfer for Large Language Model (LLM) serving. The project aims to reduce PCIe bandwidth bottlenecks by implementing a compress-transfer-decompress pipeline for KV cache swapping operations. This report documents the completed work including code framework design, integration architecture, and benchmarking infrastructure. The implementation is currently in the code development phase and requires GPU environment testing for validation and performance evaluation.

Contents

Abstract	1
1 Introduction	1
1.1 Background and Motivation	1
1.2 Project Objectives	1
1.3 Project Scope	2
2 Related Work	3
2.1 vLLM and PagedAttention	3
2.2 cuSZp Compression Framework	3
3 System Architecture	5
3.1 Overall Design	5
3.2 Compression Pipeline Architecture	5
3.2.1 Device-to-Host (D2H) Compression Flow	6
3.2.2 Host-to-Device (H2D) Decompression Flow	6
3.3 Key Design Decisions	6
3.3.1 Asynchronous Execution	6
3.3.2 Error Handling and Fallback	7
3.3.3 Memory Management	7

4	Implementation Details	8
4.0.1	cuSZp Wrapper (C++)	8
4.0.2	Python Bindings	9
4.0.3	Compression Pipeline (Python)	9
4.0.4	Build System	10
5	Testing and Benchmarking Infrastructure	11
5.0.1	Baseline Profiling	11
5.0.2	Compression Benchmarking	11
5.0.3	Expected Performance Metrics	11
6	Completed Work	13
6.0.1	Phase 1: Environment Analysis and Baseline Setup	13
6.0.2	Phase 2: cuSZp Integration Preparation	13
6.0.3	Phase 3: Compression Pipeline Implementation	13
6.0.4	Phase 4: Benchmarking Infrastructure	14
7	Current Status	15
7.0.1	Code Completion Status	15
7.0.2	Testing Status	15
8	Challenges and Solutions	16
8.0.1	Technical Challenges	16
8.0.2	Development Challenges	17
9	Next Steps	18
9.0.1	Immediate Next Steps	18
9.0.2	Performance Evaluation	18

9.0.3	Future Enhancements	18
10	Project Timeline	20
11	Conclusion	21

Chapter 1

Introduction

1.1 Background and Motivation

Large Language Models (LLMs) have revolutionized natural language processing, but their deployment faces significant challenges, particularly in memory management and data transfer efficiency. The vLLM framework [2] addresses memory efficiency through PagedAttention, which enables efficient KV cache management by swapping blocks between GPU and CPU memory.

However, CPU-GPU data transfer over PCIe remains a bottleneck. Modern GPUs can achieve compute throughputs exceeding 1000 TFLOPS, but PCIe bandwidth is typically limited to 16-32 GB/s (PCIe 3.0/4.0). This bandwidth limitation becomes critical when KV cache blocks need to be swapped between GPU and CPU memory during inference.

Error-bounded lossy compression offers a promising solution. By compressing data before transfer and decompressing after transfer, we can effectively reduce the amount of data transmitted over PCIe, potentially achieving 2-10x compression ratios while maintaining acceptable accuracy for LLM inference.

1.2 Project Objectives

The primary objectives of this project are:

1. Integrate cuSZp [1], an ultra-fast GPU error-bounded lossy compression framework, into vLLM's CPU-GPU swap mechanism
2. Implement a compress-transfer-decompress pipeline for KV cache blocks

3. Evaluate the performance improvements in terms of transfer time reduction and overall inference throughput
4. Maintain compatibility with existing vLLM features while ensuring acceptable accuracy loss

1.3 Project Scope

This project focuses on:

- Modifying vLLM's swap operations to use compression
- Creating a C++ wrapper for cuSZp with Python bindings
- Implementing asynchronous compression/decompression using CUDA streams
- Developing benchmarking infrastructure for performance evaluation

Chapter 2

Related Work

2.1 vLLM and PagedAttention

vLLM [2] is a high-throughput LLM serving framework that implements PagedAttention, a memory management technique inspired by virtual memory and paging in operating systems. PagedAttention allows non-contiguous storage of KV cache blocks, enabling efficient memory utilization and dynamic allocation.

The framework uses swap operations to move KV cache blocks between GPU and CPU memory:

- `swap_out_blocks_to_host`: Transfers blocks from GPU to CPU
- `swap_in_blocks_from_host`: Transfers blocks from CPU to GPU

These operations are critical bottlenecks when GPU memory is limited and frequent swapping is required.

2.2 cuSZp Compression Framework

cuSZp [1] is a GPU-accelerated error-bounded lossy compression framework designed for scientific computing. Key features include:

- **High Performance:** Compression speeds of 300-400 GB/s and decompression speeds of 400-600 GB/s on A100 GPUs
- **Error Bounded:** Guarantees compression errors within specified bounds (absolute or relative)

- **Multiple Encoding Modes:** Supports fixed, plain, and outlier encoding modes for different data characteristics
- **Asynchronous Execution:** Supports CUDA streams for overlapping compression with computation

The framework is particularly suitable for this project because:

1. It operates entirely on GPU, avoiding CPU-GPU transfers for compression operations
2. It provides configurable error bounds suitable for LLM inference accuracy requirements
3. It achieves high compression ratios (2-10x) for smooth data typical of neural network activations

Chapter 3

System Architecture

3.1 Overall Design

The integration follows a modular architecture with three main components:

1. **cuSZp Wrapper (C++)**: Low-level interface to cuSZp library with PyTorch tensor support
2. **Compression Pipeline (Python)**: High-level compression manager integrated with vLLM
3. **Benchmarking Infrastructure**: Performance testing and evaluation tools

3.2 Compression Pipeline Architecture

The compression pipeline modifies vLLM's swap operations as follows:

3.2.1 Device-to-Host (D2H) Compression Flow

Algorithm 1 D2H Compressed Swap

Require: GPU KV cache block, compression configuration

Ensure: Compressed data in CPU memory

- 1: Extract block from GPU KV cache
 - 2: Allocate compression buffer on GPU
 - 3: Compress block on GPU (asynchronous, using CUDA stream)
 - 4: Transfer compressed data to CPU (reduced data size)
 - 5: Store compressed data in CPU memory
 - 6: **if** compression fails **then**
 - 7: Fallback to uncompressed swap
 - 8: **end if**
-

3.2.2 Host-to-Device (H2D) Decompression Flow

Algorithm 2 H2D Compressed Swap

Require: Compressed data in CPU memory

Ensure: Decompressed block in GPU KV cache

- 1: Read compressed data from CPU memory
 - 2: Transfer compressed data to GPU (reduced data size)
 - 3: Allocate decompression buffer on GPU
 - 4: Decompress data on GPU (asynchronous, using CUDA stream)
 - 5: Write decompressed block to GPU KV cache
 - 6: **if** decompression fails **then**
 - 7: Fallback to uncompressed swap
 - 8: **end if**
-

3.3 Key Design Decisions

3.3.1 Asynchronous Execution

Compression and decompression operations are performed asynchronously using CUDA streams, allowing overlap with:

- Data transfers (using pinned memory)
- Other GPU computations

- CPU-side operations

This design maximizes GPU utilization and minimizes overall latency.

3.3.2 Error Handling and Fallback

The implementation includes robust error handling:

- Automatic fallback to uncompressed swap if compression/decompression fails
- Configurable error bounds to balance compression ratio and accuracy
- Logging and monitoring for debugging and optimization

3.3.3 Memory Management

The wrapper implements memory pool management for compression buffers to:

- Reduce allocation overhead
- Minimize memory fragmentation
- Support efficient reuse of buffers

Chapter 4

Implementation Details

4.0.1 cuSZp Wrapper (C++)

The C++ wrapper (`cuszp_wrapper.h/cpp`) provides a clean interface to cuSZp:

```

1 class CuSZpWrapper {
2 public:
3     struct CompressionConfig {
4         float error_bound = 1e-4f;
5         bool use_relative_error = true;
6         cuszp_dim_t processing_dim = CUSZP_DIM_1;
7         cuszp_mode_t encoding_mode = CUSZP_MODE_PLAIN;
8         cuszp_type_t data_type = CUSZP_TYPE_FLOAT;
9     };
10
11     bool compress(
12         torch::Tensor input_tensor,
13         torch::Tensor& compressed_buffer,
14         size_t& compressed_size,
15         cudaStream_t stream = nullptr
16     );
17
18     bool decompress(
19         torch::Tensor compressed_buffer,
20         size_t compressed_size,
21         torch::Tensor output_tensor,
22         cudaStream_t stream = nullptr
23     );
24 };

```

Listing 4.1: CuSZpWrapper Class Interface

4.0.2 Python Bindings

Pybind11 is used to expose the C++ wrapper to Python:

```

1 PYBIND11_MODULE(cuszp_wrapper_cpp, m) {
2     py::class_<CuSZpWrapper::CompressionConfig>(m, "CompressionConfig")
3         .def(py::init<>())
4         .def_readwrite("error_bound", ...)
5         .def_readwrite("encoding_mode", ...);
6
7     py::class_<CuSZpWrapper>(m, "CuSZpWrapper")
8         .def(py::init<...>())
9         .def("compress", ...)
10        .def("decompress", ...);
11 }

```

Listing 4.2: Python Binding Example

4.0.3 Compression Pipeline (Python)

The CompressedSwapManager class manages compressed swap operations:

```

1 class CompressedSwapManager:
2     def __init__(self, enable_compression=True,
3                 error_bound=1e-4, encoding_mode="plain"):
4         # Initialize cuSZp wrapper
5         config = CompressionConfig(...)
6         self.compressor = CuSZpWrapper(config, device_id)
7         self.compression_stream = torch.cuda.Stream()
8         self.decompression_stream = torch.cuda.Stream()
9
10    def swap_out_blocks_to_host_compressed(self, ...):
11        # Compress on GPU, transfer to CPU
12        with torch.cuda.stream(self.compression_stream):
13            compressed_size = ctypes.c_size_t(0)
14            success = self.compressor.compress(...)
15            compressed_cpu = compressed_buffer.cpu()
16
17    def swap_in_blocks_from_host_compressed(self, ...):
18        # Transfer from CPU, decompress on GPU
19        compressed_gpu = compressed_cpu.to(device)
20        with torch.cuda.stream(self.decompression_stream):
21            success = self.compressor.decompress(...)

```

Listing 4.3: CompressedSwapManager Core Methods

4.0.4 Build System

The project uses CMake for building:

- `cuSZp/CMakeLists.txt`: Builds cuSZp library
- `integration/cuszp-wrapper/CMakeLists.txt`: Builds wrapper and Python bindings
- `build.sh`: Automated build script for convenience

Chapter 5

Testing and Benchmarking Infrastructure

5.0.1 Baseline Profiling

The `baseline_profiling.py` script measures baseline performance:

- **H2D Transfer:** Measures host-to-device transfer bandwidth
- **D2H Transfer:** Measures device-to-host transfer bandwidth
- **Async Overlap:** Evaluates asynchronous transfer and computation overlap

5.0.2 Compression Benchmarking

The `compression_benchmark.py` script evaluates compression performance:

- **Compression Speed:** Measures compression throughput (GB/s)
- **Decompression Speed:** Measures decompression throughput (GB/s)
- **Compression Ratio:** Calculates data size reduction
- **Error Analysis:** Evaluates compression errors (max, mean)
- **Error Bound Sweep:** Tests performance across different error bounds

5.0.3 Expected Performance Metrics

Based on cuSZp performance characteristics and typical PCIe bandwidth:

Table 5.1: Expected Performance Improvements

Metric	Baseline	With Compression
D2H Transfer Time	100%	30-70% (reduction)
H2D Transfer Time	100%	30-70% (reduction)
Compression Speed	N/A	300-400 GB/s
Decompression Speed	N/A	400-600 GB/s
Compression Ratio	1x	2-10x

Chapter 6

Completed Work

6.0.1 Phase 1: Environment Analysis and Baseline Setup

- Analyzed vLLM code structure and CPU-GPU swap mechanisms
- Identified integration points (`swap_out_blocks_to_host`, `swap_in_blocks_from_host`)
- Created project documentation structure
- Designed integration architecture

6.0.2 Phase 2: cuSZp Integration Preparation

- Analyzed cuSZp API and compression workflow
- Designed C++ wrapper class (`CuSZpWrapper`)
- Implemented compression and decompression interfaces
- Created Python bindings using `pybind11`
- Set up CMake build system

6.0.3 Phase 3: Compression Pipeline Implementation

- Implemented `CompressedSwapManager` class
- Created compressed versions of swap operations
- Added asynchronous execution support using CUDA streams

- Implemented error handling and fallback mechanisms
- Created automated build script (`build.sh`)

6.0.4 Phase 4: Benchmarking Infrastructure

- Created baseline performance profiling script
- Created compression performance benchmark script
- Added Docker support for containerized deployment
- Created comprehensive documentation (integration guide, deployment guide)

Chapter 7

Current Status

7.0.1 Code Completion Status

Table 7.1: Implementation Status	
Component	Status
cuSZp Wrapper (C++)	✓ Complete
Python Bindings	✓ Complete
Compression Pipeline	✓ Complete
Build System	✓ Complete
Benchmarking Scripts	✓ Complete
Documentation	✓ Complete

7.0.2 Testing Status

- **Code Framework:** All code has been written and structured
- **Compilation:** Code structure is ready for compilation (requires GPU environment)
- **Runtime Testing:** Pending GPU environment access
- **Performance Evaluation:** Benchmarking scripts ready, awaiting execution

Chapter 8

Challenges and Solutions

8.0.1 Technical Challenges

Memory Management

Challenge: Compressed data sizes vary, requiring dynamic buffer allocation.

Solution: Implemented buffer size estimation and dynamic allocation with memory pooling for efficiency.

Integration with vLLM

Challenge: vLLM's CPU cache structure expects uncompressed data.

Solution: Designed wrapper that can store compressed data with metadata, with fallback to uncompressed storage if needed.

Asynchronous Execution

Challenge: Ensuring proper synchronization between compression, transfer, and decompression.

Solution: Used CUDA streams with proper synchronization points and error checking.

8.0.2 Development Challenges

GPU Environment Access

Challenge: Limited access to GPU environments for testing during development.

Solution: Structured code to be testable in stages, with comprehensive error handling and logging for remote debugging.

Chapter 9

Next Steps

9.0.1 Immediate Next Steps

1. **GPU Environment Setup:** Obtain access to GPU server or cloud GPU instance
2. **Compilation and Testing:** Compile cuSZp and wrapper, verify basic functionality
3. **Unit Testing:** Test compression/decompression with various data patterns
4. **Integration Testing:** Integrate with vLLM and test end-to-end functionality

9.0.2 Performance Evaluation

1. Run baseline profiling to establish performance metrics
2. Run compression benchmarks to measure compression performance
3. Conduct end-to-end tests with real LLM inference workloads
4. Compare performance with and without compression
5. Analyze accuracy impact of compression on model outputs

9.0.3 Future Enhancements

1. **Optimization:** Fine-tune compression parameters for different model architectures
2. **Memory Management:** Implement more sophisticated memory pool management
3. **Metadata Storage:** Design efficient storage for compression metadata

4. **Multi-GPU Support:** Extend support for multi-GPU environments
5. **Dynamic Adaptation:** Implement adaptive compression based on workload characteristics

Chapter 10

Project Timeline

Table 10.1: Project Timeline

Phase	Duration	Status
Phase 1: Environment Analysis	Weeks 1-3	✓ Complete
Phase 2: cuSZp Integration Prep	Weeks 4-6	✓ Complete
Phase 3: Pipeline Implementation	Weeks 7-11	✓ Complete
Phase 4: Benchmarking Framework	Weeks 12-15	✓ Complete
Phase 5: Testing & Evaluation	Weeks 16-20	→In Progress
Phase 6: Optimization & Analysis	Weeks 21-24	Pending

Chapter 11

Conclusion

This interim report documents the substantial progress made in integrating cuSZp compression into vLLM's CPU-GPU data pipeline. The code framework is complete, including:

- C++ wrapper for cuSZp with PyTorch integration
- Python compression pipeline integrated with vLLM architecture
- Comprehensive benchmarking infrastructure
- Complete build system and documentation

The project is now ready for GPU environment testing and performance evaluation. The modular architecture and comprehensive error handling provide a solid foundation for optimization and refinement based on empirical results.

The next phase will focus on validating the implementation, measuring performance improvements, and optimizing compression parameters for real-world LLM serving scenarios.

References

- [1] Yafan Huang, Sheng Di, Xiaodong Yu, Guanpeng Li, and Franck Cappello. cuszp: An ultra-fast gpu error-bounded lossy compression framework with optimized end-to-end performance. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–13, 2023.
- [2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.